📄 PROJECT_RULES.md

> **Project:** Prism IQ
>
> **Purpose:** Engineering Constitution
>
> **Version:** 1.0
>
> **Status:** Mandatory

SECTION A — Core Philosophy

Rule 1

Prism IQ is an **AI-assisted analytics workspace**, not a dashboard clone.

Rule 2

Every new feature must support the analytics pipeline.

Rule 3

Never sacrifice architecture for speed.

Rule 4

Prefer simplicity over unnecessary abstraction.

Rule 5

Optimize for maintainability over clever code.

Rule 6

Every architectural decision must align with the ADR document.

Rule 7

If documentation and implementation disagree, update the documentation or discuss the change before coding.

SECTION B — Data Integrity

Rule 8

Original uploaded datasets are immutable.

Never modify them.

Rule 9

Cleaning always produces a new cleaned dataset.

Rule 10

Every transformation must be reproducible.

Rule 11

Never silently discard rows.

Report all removals.

Rule 12

Every cleaning operation must produce a summary.

Rule 13

Downloaded cleaned files must always correspond to the recorded cleaning operations.

SECTION C — Architecture

Rule 14

Routes never contain business logic.

Rule 15

Services own business logic.

Rule 16

Repositories only interact with PostgreSQL.

Rule 17

Schemas are never reused as ORM models.

Rule 18

Utilities must remain stateless.

Rule 19

Every module should have one responsibility.

Rule 20

No circular imports.

SECTION D — APIs

Rule 21

All APIs return the standard response wrapper.

Rule 22

Use correct HTTP status codes.

Rule 23

Validate all inputs before processing.

Rule 24

Never expose internal exceptions to clients.

Rule 25

Version APIs using `/api/v1`.

Rule 26

Breaking changes require a new API version.

SECTION E — Database

Rule 27

PostgreSQL stores metadata only.

Rule 28

Datasets remain outside the database.

Rule 29

UUIDs are the default primary key.

Rule 30

Use migrations for schema changes.

Never modify production schema manually.

Rule 31

Database access must go through repositories.

SECTION F — Machine Learning

Rule 32

Model selection is automatic.

Rule 33

Users select only the target column.

Rule 34

The backend decides the problem type.

Rule 35

Compare multiple supported models before selecting the best one.

Rule 36

Return interpretable metrics.

Rule 37

Never expose raw sklearn objects through APIs.

SECTION G — Reports

Rule 38

Reports prioritize decisions over raw numbers.

Rule 39

Recommendations must be actionable.

Rule 40

Reports remain concise.

Rule 41

Charts included in reports must also be visible in the UI.

SECTION H — Frontend

Rule 42

The UI follows the Design System.

Rule 43

No inline styles for production components.

Rule 44

No hardcoded colors.

Use design tokens.

Rule 45

Every page supports loading, error, and empty states.

Rule 46

All reusable UI belongs in shared components.

Rule 47

Pages orchestrate components; they do not duplicate UI logic.

Rule 48

Accessibility is considered from the beginning.

SECTION I — User Experience

Rule 49

Every page answers:

- Where am I?
- What happened?
- What should I do next?

Rule 50

One primary action per screen.

Rule 51

Avoid overwhelming users with statistics.

Rule 52

Guide users through the pipeline.

Rule 53

The product should explain results, not only display them.

SECTION J — Security

Rule 54

Validate uploaded files.

Rule 55

Validate URLs before scraping.

Rule 56

Reject unsupported formats.

Rule 57

Never trust client input.

Rule 58

Sanitize filenames.

Rule 59

Never log sensitive information.

SECTION K — Performance

Rule 60

Avoid repeated DataFrame scans.

Rule 61

Prefer vectorized Pandas operations.

Rule 62

Avoid unnecessary recomputation.

Rule 63

Process only the required columns when possible.

SECTION L — Logging

Rule 64

Every major operation must be logged.

Rule 65

Errors always include contextual information in logs.

Rule 66

Use structured logging where practical.

SECTION M — Testing

Rule 67

Every service requires unit tests.

Rule 68

Critical workflows require integration tests.

Rule 69

Bug fixes should include regression tests when feasible.

Rule 70

Features are incomplete if tests fail.

SECTION N — Git

Rule 71

Commit messages follow Conventional Commits.

Rule 72

Keep commits focused.

Rule 73

Do not mix refactoring with feature work in one commit unless tightly related.

Rule 74

Documentation updates accompany architectural changes.

SECTION O — Documentation

Rule 75

Major decisions require an ADR entry.

Rule 76

APIs must remain synchronized with the API Specification.

Rule 77

MASTER_CONTEXT.md is the source of truth for AI sessions.

Rule 78

Design changes update the UI/UX document.

SECTION P — AI Rules

Rule 79

AI must read `MASTER_CONTEXT.md` before implementation.

Rule 80

AI must never redesign the architecture unless explicitly instructed.

Rule 81

AI should extend existing modules before creating new ones.

Rule 82

AI should preserve naming conventions.

Rule 83

AI should not modify unrelated files.

Rule 84

AI should produce reviewable changes instead of massive rewrites.

Rule 85

Generated code should match the project's coding standards.

SECTION Q — Code Quality

Rule 86

No TODOs in merged production code unless tracked.

Rule 87

Remove dead code.

Rule 88

Avoid duplicated logic.

Rule 89

Prefer composition over inheritance unless inheritance is clearly justified.

Rule 90

Magic numbers should be replaced with named constants.

Rule 91

Public functions require type hints and documentation.

Rule 92

Long methods should be split into smaller units.

SECTION R — Product Scope

Rule 93

The MVP includes only documented features.

Rule 94

Feature creep requires explicit approval.

Rule 95

Future ideas belong in the roadmap, not the current sprint.

SECTION S — Definition of Done

A task is complete only if:

- Feature implemented.
- Backend completed.
- Frontend integrated.
- Validation added.
- Errors handled.
- Loading states added.
- Empty states added.
- Tests pass.
- Documentation updated.
- Code reviewed.

SECTION T — Engineering Mindset

Rule 96

Build systems, not hacks.

Rule 97

Optimize for clarity before optimization.

Rule 98

Every feature should improve user value.

Rule 99

Every contribution should leave the codebase cleaner than before.

Rule 100

When in doubt, choose the solution that is easiest to understand, test, and maintain.

🔒 AI Constitution

Every AI assistant contributing to Prism IQ must follow this pledge:

> I will respect the documented architecture, preserve data integrity, maintain consistency with the design system, follow the coding standards, avoid unnecessary rewrites, and prioritize maintainable, production-quality code over shortcuts.

📌 Document Hierarchy (Priority Order)

To avoid conflicts, every contributor should follow this precedence:

1. **PROJECT_RULES.md** *(Highest authority — engineering constitution)*
2. **MASTER_CONTEXT.md** *(Project vision and architecture)*
3. **ARCHITECTURE_DECISIONS.md** *(Reasons behind major decisions)*
4. **CODING_STANDARDS.md** *(Implementation style and conventions)*
5. **API_SPECIFICATION.md** *(Backend contract)*
6. **UI_UX.md** *(Design system and UX guidelines)*
7. **TRD.md** *(Technical requirements)*
8. **PRD.md** *(Business and product requirements)*

If two documents appear to conflict, the one with higher priority takes precedence unless a new architectural decision is formally approved.
